CS242 : System Software Lab

Debugger: gdb and ddd

A Sahu

Dept of CSE, IIT Guwahati

What are Errors?

- Syntax Errors
  - Compiler/Linker catch these
  - Mistakes in your formatting of C/C++
- Semantic/Logic Errors
  - Nothing catches these
  - Misunderstanding of programmer about what system is supposed to do
    - These are BUGS
    - Mismatch between what system is supposed to do and what it actually does

Finding Bugs

- Categories of Bugs
  - Seg-fault or core-dump (fatal!)
  - Program infinitely loops
  - Runs but output is incorrect
- Strategies
  - Look through code line by line
  - Print values every once in a while
  - Use a debugger (best choice!)
- Professional Programmers?
  - Use a mix of these strategies!

Debuggers on Linuix

- GDB
  - "GNU DeBugger"
  - Text-based
  - Fast to load
- DDD
  - "GNU Data Display Debugger"
  - Graphically based
  - Easier to use
  - Slower to load/interact with (remotely)
  - Must install an GUI enabled server

Old-Fashioned Debugging

- The point of debugging is to find your errors
- Simplest technique is **checkpointing**
  - Place an output statements around calls to subroutines
    - Printf(“Entering subroutine A”)
    - A();
    - Printf(“Completed subroutine B”)
  - If your program crashes in A, you won’t see the second line
- Work into subroutines, bracketing sections of code with outputs until you find where the error occurs.

Old-Fashioned Debugging

- Checkpointing is nice because it works on any system that can run your code
- But, requires lots of compiles as you zero in on bug.
- Can also output data values
- WARNING: Finding the line where the program crashes is not enough, you need to know why!
  - The problem could result from a previous statement
  - In this case, figure out where the variables on the offending line are set, and work backwards

Debugging

- UNIX/Linux standard debugging program is gdb
- gdb allows you to watch your program run
  - Set breakpoint--position in code, execution will stop when reached
  - Step through program line-by-line
  - Examine value of variables

GDB/DDD – Allows you to do

- Run program from start
- See which line seg-faulted
- Run program line by line
- Stop at any point
- Print variables at any point
- View parameters
- Trace through function calls
- Exit
- Get Help on any feature

GDB: Requirement

- To use gdb, compile with -g flag
  - On most systems, can't use -g with -O (optimizations)
- Then type
  - $gdb program inputs
- GDB will start and it will "grab" your program

GDB: Typical Session

- Set breakpoint(s) (br)
- Run until you hit a breakpoint (run)
- Step through some lines (n, s)
- Look at some variables (p)
- Continue to next breakpoint (c)
- Quit (q)

GDB: Setting Breakpoint

- break ReadComm-- sets a breakpoint at ReadComm
- break io.c:21 --sets a breakpoint at line 21 in io.c

GDB: Stepping through

- Typing run will take you to the next breakpoint
- You can step through line by line by typing n(ext)
  - gdb will display the next line of code to be executed
- If the next line is a call to another subroutine
  - Typing s(tep) will “step into” that rountine
  - Typing n(ext) will skip to the next routine

GDB: Viewing Variables

- Viewing variables
  - You can check the value of a variable by typing “p name”
    - This may not be what you expect
      - p array will give the memory address of the array
      - p array[0] will give the value at the first location
- gdb is complicated
  - Type $man gdb to see more info

GDB Basic Commands

Command	Abbreviation	Description
gdb [executable]		Starts gdb and loads the executable
run [cmdLinParms]	r cmdLineParms	Runs the loaded executable
list [point]	l l lineNbr l File:lineNbr l function	Lists several lines of code around/at a point. Points can be line numbers, function names, or lines in a particular file, absence of a point indicates "next few lines"
break [point]	b b lineNbr b function	Sets a breakpoint at a point. This will stop execution at this point. You can then view variables at that point or perform other tasks.
continue	c	Run until next breakpoint or end
print variable print function	p variableName p functionCall	Prints the value of a variable or the return value from a function call at the current line.
printf formatting var/func		Works just like printf in C.

GDB Basic Commands

Command	Abbreviation	Description
display var/func	disp variableName disp functionCall	Works just like print, except that it displays those values every time you stop
watch variable	wa variableName	Pause execution whenever variable changes
next	n	Runs the next line of code, skips over functions
step	s	Runs to first line of code inside a function call
backtrace where up down	bt	Allows you to see function call sequence that led to current line of code. Up takes you up one level Down takes you down one level
quit	q	Quit gdb
help [topic]	h topic	Gets help on a particular topic, or general help

GDB: Example 1

```
#include <iostream>
using namespace std;
int divint(int, int);
int main() {
        int x = 5, y = 2;
        cout << divint(x, y);
        x =3; y = 0;
        cout << divint(x, y);
        return 0;
 }
 int divint(int a, int b) {
 return a / b;
 }
```

GDB: Example 1

- $g++ -g crash1.cpp -o crash1
- $gdb crash1
- (gdb) run
  - Program received signal SIGFPE, Arithmetic exception. 0x08048681 in divint(int, int) (a=3, b=0) at crash1.cpp :21 21 return a / b;
- (gdb) list
- (gdb) where
- (gdb) p x
- How to run core file $ ./crash1; gdb crash1 core

GDB: Example 2

```
#include <iostream>
using namespace std;
void setint(int*, int);
int main() {
        int a;
        setint(&a, 10);
        cout << a << endl;
        int* b;
        setint(b, 10);  //Mistake is intentional to check gdb demo
        cout << *b << endl;
        return 0;
}
void setint(int* ip, int i) {
*ip = i;
}
```

GDB: Example 2

- $g++ -g crash2.cpp -o crash2
- $gdb crash2
- (gdb) run
  - Program received signal SIGSEGV, Segmentation fault. 0x4000b4d9 in _dl_fini () from /lib/ld-linux.so.2
  - (gdb) where
- How to run core file $ ./crash1; gdb crash1 core

GDB: Example 2

- (gdb) b main
  - # Set a breakpoint at the beginning of the function main
- (gdb) r
  - # Run the program, but break immediately to the breakpoint.
- (gdb) n
  - # n = next, runs one line of the program
- (gdb) n
- (gdb) s
  - setint(int*, int) (ip=0x400143e0, i=10) at crash2.cpp:20 # s = step, is like next, but it will step into functions. (gdb) p ip
  - $3 = (int *) 0x400143e0
- (gdb) p *ip
  - 1073827128

GDB: Example 3

```
/* File: buggy.c */
 #include <stdio.h>
int main() {
int balance=100;
int target=1000;
float rate = 0.1;
int year = 0;
do {
        float interest = balance * rate;
        balance = balance + interest;
        year++;
} while ( balance >= target );   //intensional error
printf("%d No. of years to achieve target balance.\n", year);
return 0;
}
```

GDB: Example 3

- (gdb) b main
  - Breakpoint 1 at 0x400535: file buggy.c, line 5
- (gdb) run
  - # Run the program, but break immediately to the breakpoint.
- (gdb) s
  - 6 int target=1000;
- (gdb) s. #do it upto line 13
  - ……
- (gdb) p target
- (gdb) p balance
- (gdb) p year
  - balance >= target, will be false, loop get break
- (gdb) p &year
  - 0xbffff580
- (gdb) x 0xbffff580

GDB: Example 4

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

char * buf;

int sum_to_n(int num) {
 int i,sum=0;
for(i=1;i<=num;i++) sum+=i;
return sum;
}

int main() {
printSum();
return 0;
}
```

```
void printSum() {
char line[10];
printf("enter a number:\n");
fgets(line, 10, stdin);
if(line != NULL)
strtok(line, "\n");
sprintf(buf,"sum=%d",sum_to_n(at
oi(line)));
printf("%s\n",buf);
 }
```

GDB: Example 4

- (gdb) run
  - Program received signal SIGSEGV, Segmentation fault.
    0x0017fa24 in _IO_str_overflow_internal () from /lib/tls/libc.so.6.
- (gdb) backtrace
  - #0 0x0017fa24 in _IO_str_overflow_internal () from /lib/tls/libc.so.6
    #1 0x0017e4a8 in _IO_default_xsputn_internal () from /lib/tls/libc.so.6
    #2 0x001554e7 in vfprintf () from /lib/tls/libc.so.6
    #3 0x001733dc in vsprintf () from /lib/tls/libc.so.6
    #4 0x0015e03d in sprintf () from /lib/tls/libc.so.6
    #5 0x08048487 in printSum () at crash.c:22
    #6 0x080484b7 in main () at crash.c:28
- (gdb) beak crash.c 22
- (gdb) run
- (gdb) print line
- (gdb) print buf
-
  -

Modern Debugging: with DDD

- IDE's like VizStudio have graphical debuggers
- DDD: Data Display debugger
  - On some systems, this is just a GUI for db
- DDD
  - "GNU Data Display Debugger"
  - Graphically based
  - Easier to use
  - Slower to load/interact with (remotely)
  - Must install an GUI enabled server
- See the demo